



Universidade Federal da Paraíba
Centro de Energias Alternativas e Renováveis

RELATÓRIO SENSORIAMENTO: Semana 1

Rayanne Maria Lucena de Oliveira
20230047177

João Pessoa-PB
2026

LISTA DE CÓDIGOS

1	Código Teste no Arduino	8
2	Peceptor em Python	8

SUMÁRIO

1	Caio	3
1.1	Informações Gerais.....	3
1.2	Objetivo da Sprint S1	3
1.3	Atividades Realizadas	3
1.3.1	<i>Correção da Interface LoRa — Pinos AUX</i>	<i>3</i>
1.3.1.1	<i>Ações tomadas.....</i>	<i>3</i>
1.3.2	<i>Configuração de Endereço de Transmissão Fixo</i>	<i>4</i>
1.3.2.1	<i>Resultado obtido</i>	<i>4</i>
1.3.3	<i>Início do Projeto da PCB Receptora no KiCad</i>	<i>4</i>
1.3.3.1	<i>Estado atual.....</i>	<i>4</i>
1.4	Status dos Entregáveis — S1	4
1.5	Impedimentos e Riscos Identificados.....	5
1.5.1	<i>Risco mapeado no planejamento — Falha na comunicação LoRa:</i> <i>.....</i>	<i>5</i>
1.5.2	<i>Pendência — Script de leitura serial:.....</i>	<i>5</i>
1.6	Próximos Passos — Sprint S2 (até 28/04/2026).....	5
2	Fernando	5
2.1	Objetivos da Etapa	5
2.2	Mapeamento do Barramento CAN (Sevcon Gen4)	5
2.2.1	<i>Análise de Dados.....</i>	<i>5</i>
2.3	Definição da Pilha de Software	6
2.4	Detalhamento da Pinagem e Conexões (ESP32).....	6
2.4.1	<i>Especificação dos Sinais</i>	<i>6</i>
2.4.1.1	<i>CAN TX (Transmit) — GPIO 5</i>	<i>6</i>
2.4.1.2	<i>CAN RX (Receive) — GPIO 4.....</i>	<i>6</i>
2.4.2	<i>Tabela de Pinagem — ESP32 (CAN Interface).....</i>	<i>6</i>
2.4.3	<i>Requisitos Elétricos.....</i>	<i>6</i>
2.5	Conclusão e Próximos Passos.....	6
2.5.1	<i>Próximo marco:</i>	<i>7</i>
3	Leo	7
3.1	Arquivos.....	7
3.1.1	<i>Dashboard_{v2}</i>	<i>7</i>
3.1.2	<i>Código Teste Arduino</i>	<i>7</i>
3.2	Desenvolvimento do Dashboard em Qt/C++	7
4	Rayanne	8
4.1	Atividades	8
4.1.1	<i>Comunicação Arduino</i>	<i>8</i>
4.1.1.1	<i>Resultado</i>	<i>9</i>
4.1.2	<i>Teste com Executável.....</i>	<i>9</i>
4.1.2.1	<i>Resultado</i>	<i>9</i>
4.1.3	<i>Teste com HTML.....</i>	<i>10</i>
4.1.3.1	<i>Resultado</i>	<i>10</i>

1 CAIO

1.1 Informações Gerais

Quadro 1 – Informações Gerais

Data do Relatório	02/05/2026
Sprint	S1 — Semana 1
Período de Referência	14/04/2026 a 21/04/2026
Marco da Semana	Comunicação LoRa validada / Layout PCB receptora iniciado
Responsável	Caio (Supervisor) e Leonardo (Membro)
Subsistema	Unidade de Recepção e Software (Subsistema 2)
Data-limite do Marco S1	21/04/2026

Fonte: elaborado pelo autor (2026)

1.2 Objetivo da Sprint S1

Conforme o planejamento do projeto, o marco da Semana 1 para o Subsistema 2 (PCB Receptora e Software) previa:

- Início do esquemático da PCB receptora no KiCad;
- Desenvolvimento e validação de um script de leitura serial no PC com dados simulados no formato do payload (RPM:3500;VEL:72;TQ:120), sem dependência de hardware físico;
- Garantir base sólida de comunicação LoRa para integração nas semanas seguintes.

1.3 Atividades Realizadas

1.3.1 Correção da Interface LoRa — Pinos AUX

Durante os testes iniciais do módulo LoRa Ebyte E32-433T20D, foram identificados erros de comunicação entre o ESP32 e o Arduino. Após diagnóstico, constatou-se que o pino AUX do módulo LoRa não estava sendo utilizado corretamente no firmware. O pino AUX é responsável por sinalizar ao microcontrolador quando o módulo está pronto para enviar ou receber dados; sem ele, ocorriam colisões e falhas silenciosas na transmissão.

1.3.1.1 Ações tomadas :

- Revisão da datasheet do módulo Ebyte E32 para mapeamento correto dos pinos M0, M1 e AUX;
- Refatoração do código de inicialização do LoRa para monitorar o estado do pino AUX antes de qualquer operação de leitura ou escrita;
- Validação funcional após a correção: comunicação estabelecida com sucesso entre os dois módulos na bancada.

1.3.2 Configuração de Endereço de Transmissão Fixo

Para garantir interoperabilidade entre diferentes placas Arduino utilizadas pela equipe, foi definido e implementado um endereço de transmissão fixo no firmware LoRa. Com essa padronização, qualquer Arduino equipado com um módulo Ebyte E32 e o código padrão da equipe é capaz de se conectar e receber as informações transmitidas pelo ESP32 embarcado no carro, sem necessidade de reconfiguração manual.

1.3.2.1 Resultado obtido :

- Endereço de transmissão (ADDH/ADDL) e canal de comunicação fixados no firmware;
- Teste realizado com sucesso entre ESP32 (transmissor) e Arduino (receptor) na mesma configuração de endereço;
- Código versionado e documentado para replicação em outras placas da equipe.

1.3.3 Início do Projeto da PCB Receptora no KiCad

O projeto eletrônico da PCB receptora foi inicializado na ferramenta KiCad. Esta placa integra o Arduino e o módulo LoRa Ebyte E32, sendo responsável por receber os dados transmitidos pelo carro e repassá-los via porta Serial para o software de telemetria no notebook.

1.3.3.1 Estado atual :

- Esquemático iniciado com os componentes principais inseridos (Arduino, conector LoRa, alimentação);
- Mapeamento dos pinos AUX, M0, M1 e UART já definido no esquemático;
- Projeto em andamento, com layout e geração de Gerbers previstos para a conclusão da semana.

1.4 Status dos Entregáveis — S1

Quadro 2 – Status dos Entregáveis — S1

Entregável	Responsável	Status
Correção do erro de comunicação LoRa (pino AUX)	Caio / Leonardo	CONCLUÍDO
Endereço de transmissão fixo configurado	Caio / Leonardo	CONCLUÍDO
Teste de comunicação ESP32 – Arduino	Caio / Leonardo	CONCLUÍDO
Início do esquemático PCB receptora (KiCad)	Caio / Leonardo	EM ANDAMENTO
Script de leitura serial com dados simulados	Caio / Leonardo	PENDENTE

Fonte: elaborado pelo autor (2026)

1.5 Impedimentos e Riscos Identificados

1.5.1 *Risco mapeado no planejamento — Falha na comunicação LoRa:*

O erro identificado nos pinos AUX corresponde exatamente ao risco de falha na comunicação LoRa previsto no documento de planejamento. A mitigação adotada (validação isolada do LoRa antes da integração com CAN) foi executada com sucesso, e o risco foi resolvido ainda na Sprint S1.

1.5.2 *Pendência — Script de leitura serial:*

O script de leitura serial com dados simulados ainda não foi desenvolvido. Este item deve ser priorizado no início da Sprint S2 para não comprometer a integração prevista para S3.

1.6 Próximos Passos — Sprint S2 (até 28/04/2026)

- Finalizar o esquemático da PCB receptora e avançar para o layout no KiCad;
- Desenvolver e validar o script de leitura serial no PC com payload simulado (RPM:XXXX;VEL:XX;TQ:XXX);
- Implementar logging de dados em arquivo TXT/CSV;
- Iniciar fabricação local da PCB receptora (conforme planejamento de prototipagem rápida);
- Realizar teste de alcance e estabilidade do LoRa com firmware atualizado.

2 FERNANDO

2.1 Objetivos da Etapa

O objetivo desta fase foi o levantamento dos requisitos de software e o mapeamento dos identificadores (IDs) de comunicação do inversor Sevcon Gen4 Size 8 para monitoramento dos parâmetros críticos de performance.

2.2 Mapeamento do Barramento CAN (Sevcon Gen4)

Conforme o planejamento, foram identificados os IDs documentados para os três parâmetros prioritários:

- RPM do Motor: Identificado via TPDO1 no endereço 0x181
- Velocidade do Veículo: Identificado via TPDO3 no endereço 0x381
- Torque Real: Identificado via TPDO2 no endereço 0x281

2.2.1 *Análise de Dados*

- A comunicação utiliza o protocolo CANopen.
- Os dados de RPM e Velocidade devem ser filtrados e decodificados para processamento no ESP32.

2.3 Definição da Pilha de Software

Para o controle do barramento no microcontrolador ESP32 (Subsistema 1), foi definida a seguinte estrutura:

- Biblioteca: Driver nativo ESP32-TWAI-CAN (Two-Wire Automotive Interface).
- Justificativa: Garante estabilidade e suporte nativo ao controlador CAN interno do ESP32, permitindo a leitura de frames brutos.

2.4 Detalhamento da Pinagem e Conexões (ESP32)

A conexão entre o ESP32 e o barramento CAN exige um transceiver externo para converter sinais lógicos em níveis diferenciais (CANH/CANL).

2.4.1 Especificação dos Sinais

2.4.1.1 CAN TX (Transmit) — GPIO 5

- Função: Saída de dados do ESP32 para o transceiver.
- Fluxo: Envio de frames de configuração ou requisições.
- Nível Lógico: 3.3V (CMOS).

2.4.1.2 CAN RX (Receive) — GPIO 4

- Função: Entrada de dados do transceiver para o ESP32.
- Fluxo: Recebe mensagens do barramento (RPM, Torque, Velocidade).
- Nível Lógico: 3.3V (ESP32 não tolera 5V).

2.4.2 Tabela de Pinagem — ESP32 (CAN Interface)

Quadro 3 – Tabela Pinagem

Função CAN	Pino ESP32 (GPIO)	Conexão Transceiver	Direção do sinal
CAN _{TX}	GPIO 5	TXD (D)	Saída
CAN _{RX}	GPIO 4	RXD (R)	Entrada

Fonte: elaborado pelo autor (2026)

2.4.3 Requisitos Elétricos

- Transceiver:
Uso obrigatório de modelo compatível com 3.3V.
Exemplo: SN65HVD230.
- Terminação:
Resistência de 120Ω entre CANH e CANL (já presente no protoboard).

2.5 Conclusão e Próximos Passos

O mapeamento de IDs e a definição de pinagem garantem que o firmware desenvolvido nas etapas S2 e S3 seja compatível com o hardware físico.

2.5.1 *Próximo marco:*

- Validação da comunicação LoRa.
- Leitura bruta dos frames CAN.

3 LEO

3.1 Arquivos

3.1.1 *Dashboard_{v2}*

Arquivo feito no Qt com C++ para desktop. Neste arquivo temos o executável (dentro de release) com todas as dlls necessárias para a execução, também possui todo o código usado para criar o programa que lê e mostra à partir de um GUI os valores de rpm, torque e velocidade, além de poder gerar um arquivo .csv com todos os dados salvos documentados.

3.1.2 *Código Teste Arduino*

Código utilizado para gerar valores de rpm torque e velocidade, para que pudessem serem feitos os testes com o dashboard_{v2}.

3.2 Desenvolvimento do Dashboard em Qt/C++

Durante a primeira semana de desenvolvimento, foi possível concluir a estrutura principal de um dashboard utilizando a biblioteca Qt em C++. O objetivo do sistema é realizar a leitura de dados recebidos por comunicação serial e apresentar essas informações de forma dinâmica e organizada para o usuário.

Nesta etapa, foi implementada com sucesso a captura e interpretação de três variáveis principais: velocidade, torque e RPM. Esses dados são atualizados continuamente no sistema com uma taxa de variação de 100 milissegundos, permitindo acompanhamento em tempo real do desempenho monitorado.

Além da exibição numérica das variáveis, também foi desenvolvido um gráfico temporal capaz de representar as três grandezas em função do tempo. Essa funcionalidade possibilita uma análise visual mais detalhada do comportamento dos dados ao longo da execução.

Outro avanço importante foi a implementação do recurso de armazenamento das informações em arquivos no formato .csv. Essa funcionalidade garante o registro dos dados coletados, permitindo consultas posteriores, análises externas e integração com outras ferramentas de processamento.

Ao final da semana 1, o projeto já conta com:

- Comunicação serial funcional para leitura dos dados;
- Atualização em tempo real a cada 100 ms;
- Exibição de velocidade, torque e RPM na interface;
- Geração de gráficos em função do tempo;
- Exportação e salvamento dos dados em arquivos .csv.

4 RAYANNE

4.1 Atividades

4.1.1 Comunicação Arduino

Código 1 – Código Teste no Arduino

```

1 unsigned long ultimoEnvio = 0;
2
3
4 void setup() {
5     Serial.begin(9600);
6 }
7
8
9 void loop() {
10     if (millis() - ultimoEnvio >= 100) {
11         ultimoEnvio = millis();
12
13
14         float tempo = millis() / 1000.0;
15         float rpm = 3200.0 + 450.0 * sin(tempo * 0.9);
16         float vel = 68.0 + 12.0 * sin(tempo * 0.7);
17         float tq = 110.0 + 18.0 * sin(tempo * 1.2);
18
19
20         Serial.print("RPM=");
21         Serial.print(rpm, 1);
22         Serial.print(";VEL=");
23         Serial.print(vel, 1);
24         Serial.print(";TQ=");
25         Serial.println(tq, 1);
26     }
27 }

```

Fonte: elaborado pelo autor (2026)

```

1 import serial
2 import signal
3 import sys
4 import serial.tools.list_ports
5
6
7 def signal_handler(sig, frame):
8     print("CTRL-C pressionado")
9     ser.close()
10    sys.exit(0)
11
12
13 signal.signal(signal.SIGINT, signal_handler)
14
15
16 print("CTRL-C para sair")
17
18

```

```

19 ports = list(serial.tools.list_ports.comports())
20 for SerialPortName in ports:
21     print(SerialPortName)
22
23
24 try:
25     porta = input("Digite a porta serial (ex: COM3 ou /
dev/ttyUSB0): ")
26     ser = serial.Serial(str(porta), 9600)
27     print(ser.name)
28
29
30 except serial.SerialException:
31     print("Porta inexistente")
32     sys.exit(0)
33
34
35 while True:
36     linha = ser.read_until().decode('utf-8', errors='
ignore').strip()
37
38
39     if not linha:
40         continue
41
42
43     try:
44         dados = dict(item.split('=') for item in linha.
split(';'))
45         dados = {k: float(v) for k, v in dados.items()}
46
47
48         print(dados)
49
50
51     except Exception:
52         print("Linha inv lida:", linha)

```

Código 2 – Peceptor em Python

Fonte: elaborado pelo autor (2026)

4.1.1.1 Resultado :

Funcionou como esperado, exibindo as informações desejadas.

4.1.2 *Teste com Executável*

4.1.2.1 Resultado :

Não conectou com o arduíno, impossibilitando a exibição das informações.

4.1.3 Teste com HTML

4.1.3.1 Resultado :

Conectou com o arduíno, mas não exibiu nenhuma informação.